ITM SKILLS UNIVERSITY

School of Future Tech

Case Study Number 27 - Report

on

Survey Sphere — Targeted Survey Builder Powered By Artificial Intelligence

by

Abdeali Makda

150096725096

Cohort: Larry Page

Academic Year 2025–2029

Index

No.	Section	Page
1	Introduction to the Case Study	3
2	Problem Statement / Case Background (Abstract)	4
3	Case Study Design	5
4	Methods, Algorithms & Technology Applied	6
5	Implementation Details	8
6	Results and Conclusion	10
7	References	11

1. Introduction to the Case Study

In today's data-driven academic and professional environments, gathering structured feedback and insights from peers, students, faculty, and users is critical. However, most existing survey platforms are either expensive, overly complex, or lack intelligent features that streamline the creation and analysis process. This case study presents the design and implementation of SurveySphere — an AI-powered, browser-based survey intelligence platform built entirely with vanilla HTML, CSS, and JavaScript.

SurveySphere enables users to build surveys, collect responses, and analyse results — all from within a single-page web application. It integrates the Groq AI API (powered by Meta's Llama 3.3-70B model) to allow users to generate complete surveys from a plain-language prompt, and to receive AI-generated insights and recommendations from collected response data.

The platform was designed with Indian college students and faculty as a primary audience. Six pre-built survey templates covering common BTech use cases — such as campus facilities, faculty teaching quality, hackathons, and placement experiences — are included out of the box. The application is fully deployed on Netlify and accessible at surveysphere-in.netlify.app.

Key Features at a Glance

- AI-powered survey generation — describe your survey in plain English and the AI builds it instantly.
- 10 question types — Multiple Choice, Checkboxes, Short Text, Long Answer, Rating Scale, Yes/No, Dropdown, Linear Scale, Date, and Number.
- Demographic eligibility gate — restrict surveys by age, gender, location, and occupation.
- Real-time analytics — Chart.js visualisations with smart chart selection per question type.
- AI Insights — sentiment analysis, key themes, and actionable recommendations generated from response data.
- Export options — download responses as CSV or plain-text TXT files.
- Time tracking — per-question and total completion time recorded for every response.
- 6 built-in templates — pre-loaded college survey templates ready to share.
- Share links — one-click shareable URLs for any survey.
- Zero backend — all data stored in localStorage; fully deployable as a static site.

2. Problem Statement / Case Background (Abstract)

Background

Educational institutions routinely need to collect structured feedback — from students rating faculty, to organisers gathering post-event responses, to placement cells understanding the student experience. The current landscape of survey tools (Google Forms, SurveyMonkey, Typeform) presents several limitations for college-level use:

- Cost: Most professional features are behind paid plans.
- Complexity: Tools like SurveyMonkey have steep learning curves for non-technical users.
- No AI integration: Existing free tools do not offer AI-assisted question generation or response analysis.
- No demographic gating: Most tools cannot restrict survey access by age, occupation, or location.
- No local deployment: All data is stored on third-party servers, raising privacy concerns.

Abstract

This case study presents the design and full implementation of SurveySphere, an AI-powered survey intelligence platform. The system is built as a client-side single-page application using HTML5, CSS3, and vanilla JavaScript — requiring no backend server or database. All user data is persisted in the browser's localStorage.

The platform provides a drag-and-drop style survey builder supporting ten distinct question types, a real-time analytics dashboard powered by Chart.js, and deep AI integration through the Groq API. The AI layer, built on Meta's Llama 3.3-70B language model, enables two core capabilities: (1) generating a complete survey from a few lines of input, and (2) analysing collected responses to produce structured insights including sentiment scoring, key themes, and prioritised recommendations.

A demographic eligibility system allows survey creators to restrict participation by age range, gender, country, and occupation. Six pre-seeded templates tailored to Indian BTech students are included. The application is deployed as a static site on Netlify and requires no installation, making it immediately accessible to any user with a browser.

3. Case Study Design

SurveySphere is designed as a modular, single-page application with five distinct logical layers. Each layer is implemented as a separate JavaScript file with clearly defined responsibilities, ensuring separation of concerns and maintainability.

System Architecture

Layer	File / Component
Data Layer	storage.js — all localStorage read/write operations
AI Layer	ai.js — Groq API communication and prompt engineering
UI Controller	app.js — routing, dashboard, modals, app initialisation
Builder Engine	builder.js — survey creation and question management
Analytics Engine	analytics.js — Chart.js visualisations and AI insights
Survey Runner	survey.js — respondent-facing question flow
Stylesheet	css/style.css — complete design system
Entry Points	app.html, survey.html, landing.html

Application Flow

1. Landing Page (landing.html): Marketing page presenting the product value proposition. Users click 'Get Started' to enter the app.
2. Dashboard (app.html — Dashboard section): Displays all surveys and templates in a card grid. Supports live search and tab switching between 'My Surveys' and 'Templates'.
3. Builder (index.html — Builder section): Two-panel layout with a question type picker sidebar and a live canvas where question cards are rendered. Demographics configuration in the sidebar.
4. Survey Runner (survey.html): The public-facing respondent form. Loaded via share link (?id=...). Handles eligibility gating, one-question-at-a-time navigation, validation, and response submission.
5. Analytics (index.html — Analytics section): Two-panel layout. Left sidebar lists all surveys; right panel renders Chart.js charts and AI Insights for the selected survey.

Data Model

All data is stored in three localStorage keys:

- surveysphere_surveys — JSON array of all survey objects (id, title, description, questions, demographics, timestamps).
- surveysphere_responses — JSON object keyed by survey ID, each containing an array of response objects (answers, demographics, timePerQuestion, totalTime, completedAt).
- surveysphere_api_key — the user's Groq API key string.

4. Methods, Algorithms & Technology Applied

4.1 AI Survey Generation

The survey generation feature uses a structured prompt engineering approach. When the user submits the AI generation form, the application collects six parameters — title, purpose, target audience, number of questions, tone, and question types — and constructs a system + user prompt pair sent to the Groq chat completions API.

The system prompt instructs the model to act as a survey builder and return only valid JSON. The user prompt specifies all parameters in plain text. The model returns a JSON array of question objects which are parsed, validated, and stored as a complete survey. Regex extraction handles edge cases where the model may include surrounding text.

4.2 AI Response Analysis

The response analysis pipeline follows these steps:

- Data aggregation: Response data is summarised per question type. Choice questions are aggregated into frequency counts; rating/scale questions produce averages; text questions include up to five sample responses; checkbox answers are flattened and counted.
- Prompt construction: The aggregated data is formatted into a plain-text summary and sent to the model with a strict JSON schema in the prompt, specifying the exact structure of the expected output.
- Response parsing: The model returns a JSON object containing summary, sentiment (overall + score out of 100), keyThemes, recommendations, and highlights. Regex extraction and JSON.parse handle the response.
- Rendering: The parsed insights are rendered as a styled card below the charts, with sentiment colour coding (green/gold/red) and priority-tagged recommendation cards.

4.3 Smart Chart Selection Algorithm

Rather than using the same chart type for all questions, the analytics engine selects the most appropriate visualisation based on question type:

Question Type	Chart Type	Reason
Rating, Linear Scale	Vertical Bar (gradient)	Shows distribution across discrete values; warm-to-gold or blue-to-gold colour gradient encodes magnitude
Multiple Choice, Yes/No, Dropdown	Doughnut Chart	Ideal for single-select data as it visually represents share of each option
Checkbox	Horizontal Bar	Multi-select data does not sum to 100%; horizontal bars compare option popularity
Number	Vertical Bar	Frequency distribution of numeric inputs
Text, Long Answer	Text Card List	Free text cannot be aggregated; top responses are displayed as readable cards
Date	Sorted Text List	Date aggregation without bucketing is not meaningful

4.4 Technology Stack

Frontend	HTML5, CSS3 (CSS Custom Properties), Vanilla JavaScript (ES6+)
Charts	Chart.js 4.4.1 (CDN) — bar, doughnut, and horizontal bar charts
AI Model	Meta Llama 3.3-70B Versatile via Groq Cloud API
API Protocol	OpenAI-compatible REST (POST /v1/chat/completions, Bearer auth)
Data Storage	Browser localStorage — no backend or database required
Typography	Google Fonts: Sora (UI), DM Mono (numbers and code)
Deployment	Netlify (static site) — surveysphere-in.netlify.app
Version Control	GitHub — github.com/abdealimak/SurveySphere
Design System	Custom dark theme — #080808 background, #c9a84c gold accent

5. Implementation Details

5.1 File Structure

app.html	Main SPA shell — dashboard, builder, analytics sections + all modals
survey.html	Respondent-facing survey form, opened via share link
landing.html	Marketing landing page (entry point via _redirects)
css/style.css	Complete design system — 1,097 lines covering all components
js/storage.js	Data layer — all localStorage CRUD, exports, template seeding
js/ai.js	Groq API integration — testApiKey, generateSurvey, analyzeResponses
js/app.js	App controller — routing, dashboard, modals, survey actions
js/builder.js	Survey builder — question management, canvas rendering, save
js/analytics.js	Analytics engine — Chart.js charts, time chart, AI insights
js/survey.js	Survey runner — eligibility gate, question flow, submission
_redirects	Netlify redirect rule — routes / to landing.html

5.2 Key Implementation Decisions

Single-Page Application without a Framework: The entire dashboard, builder, and analytics are rendered inside one HTML file using CSS display toggling. This eliminates build tool complexity while delivering a fast, app-like experience.

Modular JavaScript with Global Objects: Each JS file exposes one global object (StorageManager, AIManager, BuilderManager) following the module object pattern. This provides clean separation without requiring ES modules or a bundler, which would complicate static deployment.

Defensive XSS Prevention: All user-generated content inserted into innerHTML is passed through escHtml() and escAttr() helper functions that replace &, <, >, and " with their HTML entity equivalents. This is applied consistently across all five JS files.

Chart Memory Management: Every Chart.js instance is stored in the analyticsCharts array. Before rendering a new set of charts, the array is iterated and chart.destroy() is called on each instance. This prevents memory leaks from orphaned canvas contexts when the user switches between surveys.

Checkbox Answer Handling: Checkbox questions store answers as arrays in localStorage. The analytics engine handles this by flattening the arrays during aggregation (Array.isArray check before iteration). The survey runner uses a toggle function that adds or removes values from the array on each click, avoiding the browser's native double-fire issue with label+input pairs.

API Key Validation Fix: The testApiKey function was corrected to use res.status === 429 || res.ok as the success condition. The previous implementation checked for non-401/403 status codes, which incorrectly passed Groq's 400 response for malformed or invalid keys.

Netlify URL Fix: The share link and preview URL construction was updated from window.location.pathname.replace('index.html', '') to window.location.origin + '/' — the former produced malformed paths like /appsurvey.html when deployed on Netlify where the pathname is /app rather than /index.html.

5.3 Survey Templates

Six templates are seeded into localStorage on first load via StorageManager.seedTemplate(). The function checks existing survey titles before inserting, ensuring templates are never duplicated across page reloads. The templates are:

- Customer Satisfaction Survey
- College Event Feedback
- Faculty Teaching Quality Feedback
- Campus Facilities & Infrastructure Survey
- Hackathon / Technical Fest Participant Survey
- Internship & Placement Experience Survey

6. Results and Conclusion

Findings

The following outcomes were observed across development and testing:

- AI Survey Generation: The Llama 3.3-70B model reliably generates well-structured surveys from brief prompts. With temperature set to 0.7, outputs are varied and contextually appropriate. The model consistently follows the specified JSON schema when the system prompt enforces strict output formatting.
- AI Response Analysis: At temperature 0.4, the model produces consistent, factual analysis of response data. Aggregating responses into frequency counts and averages before sending to the API significantly improves insight quality compared to sending raw answers.
- Chart Accuracy: Smart chart selection based on question type produces more informative visualisations than a one-size-fits-all approach. The doughnut chart for single-select questions and horizontal bar for checkboxes were particularly effective for comparing response distributions.
- Template Adoption: Pre-seeding six domain-specific templates dramatically reduces the time to first survey for new users. Templates are distinguishable from user surveys via the TEMPLATE badge and separate dashboard tab.
- Demographic Gating: The eligibility gate correctly intercepts ineligible respondents before they access the survey questions. The gate is transparent about requirements, displaying the specific criteria and the exact mismatch reason when a respondent is rejected.
- Performance: As a fully static, no-backend application, load times are under 1 second. All data operations are synchronous localStorage reads/writes, producing instantaneous UI updates.

Limitations

- localStorage is browser-scoped: Each browser/device instance has isolated data. Survey responses collected on one device are not visible on another. A production deployment would require a backend database.
- Groq free tier limits: The llama-3.3-70b-versatile model allows 30 requests per minute and 1,000 requests per day on the free tier. Heavy demo usage can exhaust the daily quota.
- No real-time collaboration: Multiple users cannot edit the same survey simultaneously.

Conclusion

SurveySphere demonstrates that a fully-featured, AI-powered survey platform can be built and deployed without any backend infrastructure using modern browser APIs and third-party AI services. The project integrates prompt engineering, data visualisation, and UX design into a cohesive product that addresses real pain points in the college survey workflow.

The architecture — a modular vanilla JavaScript SPA with localStorage persistence and Groq AI integration — proves that sophisticated web applications do not require complex frameworks or server infrastructure. The platform is immediately usable by students and faculty at Indian colleges through its deployed URL and pre-loaded templates.

Future extensions could include Firebase or Supabase integration for persistent cross-device storage, user authentication to prevent duplicate responses, real-time collaborative editing, and a mobile-responsive redesign for smartphone respondents.

7. References

6. Groq Cloud API Documentation — Rate limits, model specifications, and chat completions endpoint reference. https://console.groq.com/docs/rate-limits
7. Meta AI — Llama 3.3 Model Card. Technical overview of the Llama 3.3-70B Versatile language model used for AI generation and analysis.
8. Chart.js Documentation v4.4.1 — Bar, doughnut, and horizontal bar chart configuration. https://www.chartjs.org/docs/latest/
9. MDN Web Docs — localStorage API, Fetch API, URLSearchParams, and Web Storage specification. https://developer.mozilla.org
10. Google Fonts — Sora and DM Mono typeface documentation. https://fonts.google.com
11. Netlify Documentation — Static site deployment, _redirects file configuration, and auto-deploy from GitHub. https://docs.netlify.com
12. OWASP — Cross-Site Scripting (XSS) Prevention Cheat Sheet. Basis for the escHtml() and escAttr() sanitisation functions. https://cheatsheetseries.owasp.org
13. OpenAI API Compatibility Reference — Request/response format used by Groq's OpenAI-compatible endpoint. https://platform.openai.com/docs/api-reference/chat

Declaration

I hereby declare that this case study report on SurveySphere is an original work completed by me. All external references and resources have been duly cited. The application, its code, and the analysis presented in this report are my own work unless otherwise stated.

Abdeali Makda
Roll No: 150096725096
Cohort: Larry Page
School of Future Tech, ITM Skills University